

LAB REPORT

Lab: 02 Threat Modelling Using OWASP Threat Dragon and GitHub for Risk Assessment and Security Documentation

Submitted By:

NAME	REGISTRATION NO.
AMINA RASHID	UW-24-CY-BS-044
IRAM SHEHZADI	UW-24-CY-BS-015
SALIKA BIBI	UW-24-CY-BS-008

Group: 9

Course: Secure Software Design and Development (CYS-331L)

Submitted To: Miss Muniba Khan
Date of Lab: Tuesday, October 20, 2026

Table of Contents

1. Aim and Objectives 3

2. Scenario and DFD with Trust Boundaries 3

3. Asset Table 4

4. Threat List and STRIDE Summary 5

5. Risk Register 9

6. SDLC Mapping and Analysis 10

Analysis: 11

7. GitHub Work 12

7. Answers to Practice Questions 13

9. Conclusion 16

10. Role Sign-off Lines 16

1. Aim and Objectives

Aim: To analyze SmartCampus security risks using OWASP Threat Dragon and document threats, risks, and mitigations.

Objectives:

1. Identify system assets and classify them by CIA objectives.
2. Mark trust boundaries and entry points on a DFD.
3. Find and record threats using the STRIDE method.
4. Rate risks and propose mitigations.
5. Map security activities to SDLC phases.
6. Track threats as GitHub issues.

2. Scenario and DFD with Trust Boundaries

Scenario: The SmartCampus Online Exam and Result Portal is a university web portal where students submit assignments and view grades, teachers upload marks, and the admin office manages accounts. The system uses a third-party payment gateway for fees and an email service for notifications. It is hosted on a web server in a DMZ, connecting to an internal application server, database, and file storage.

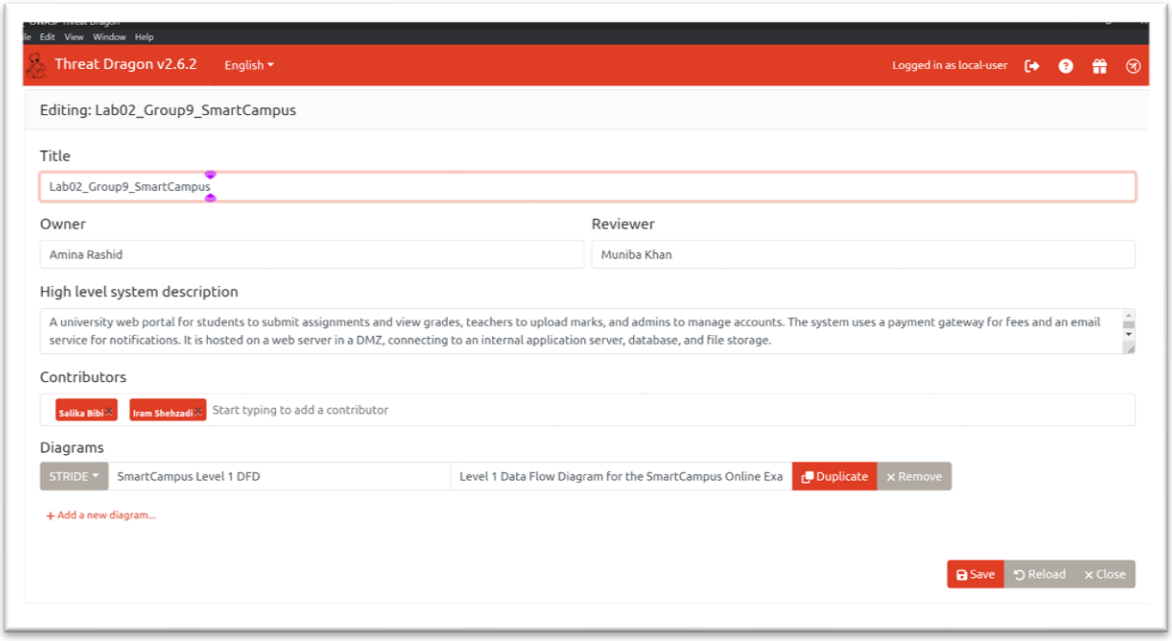


Figure 1: *Threat Dragon Model Details for Lab02_Group9_SmartCampus.*

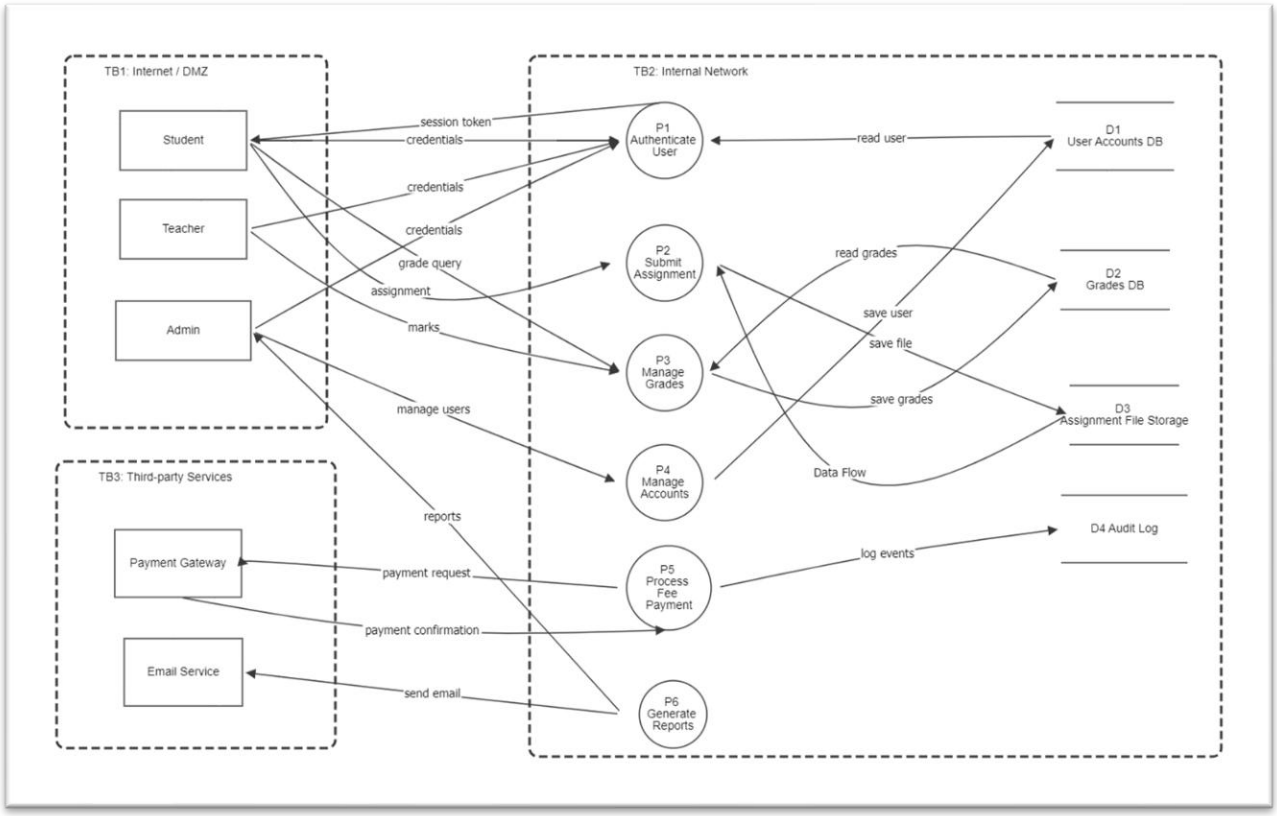


Figure 2: *SmartCampus Level 1 DFD with three Trust Boundaries (TB1, TB2, TB3).*

3. Asset Table

ID	Asset	Where it lives (DFD element)	Objective (C / I / A)	Value
A1	User passwords (hashed)	D1 User Accounts DB	C, I	High
A2	Student grades	D2 Grades DB	I, C	High
A3	Session tokens	Flow between Student and P1	C, I	High
A4	Assignment files	D3 File Storage	C, I	Medium
A5	Audit log records	D4 Audit Log	I	Medium
A6	Payment confirmation data	Flow between P5 and Gateway	I, C	High

A7	Portal uptime	P3 Manage Grades, web server	A	High
A8	Admin OTPs	Flow between Admin and P1	C	High
A9	Teacher notes	D2 Grades DB	C, I	Medium
A10	Web server config	TB2 Internal Network	C, I	High

4. Threat List and STRIDE Summary

A total of **12 threats** were identified in the SmartCampus DFD.

STRIDE :

- **Spoofing:** 2 threats (Brute-force login, Default Admin password)
- **Tampering:** 4 threats (Fake payment, SQL injection, File alteration, Malware upload)
- **Repudiation:** 1 threat (Marks changed without audit trail)
- **Information Disclosure:** 2 threats (Session token theft, Unencrypted DB backup)
- **Denial of Service:** 2 threats (Result-day flood, Report generation crash)
- **Elevation of Privilege:** 1 threat (Altering grade ID in URL)

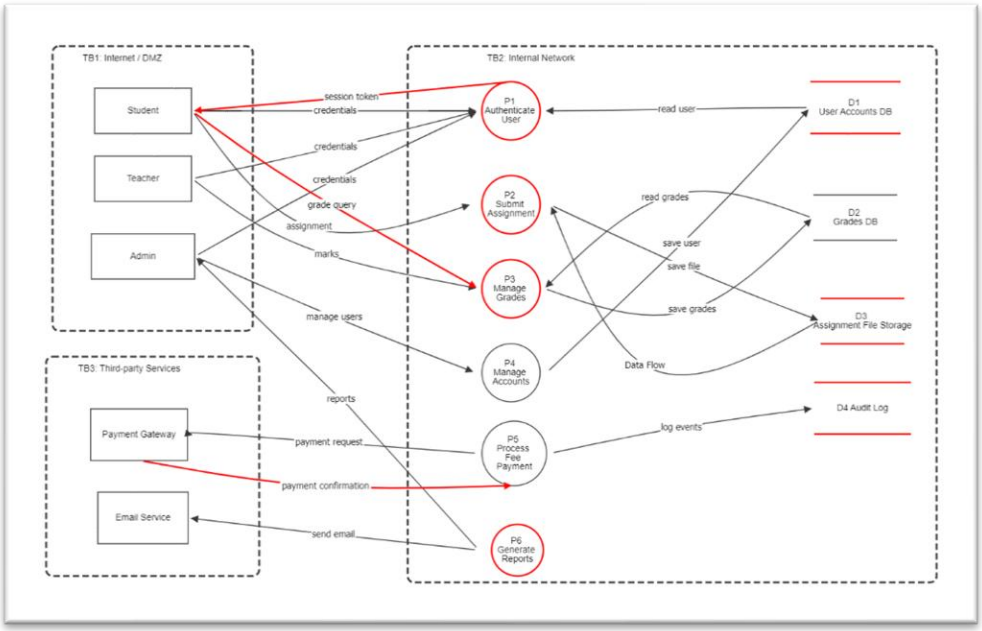


Figure 3: *Threat Dragon list of all 12 threats.*

P1 Authenticate User (Process)

Description: Validates credentials. Students/Teachers use password; Admins use password + OTP.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Brute-force attack on teacher login	Spoofing	High	Open	16	An attacker runs an automated tool against the login page to guess a teacher's password. The login allows unlimited attempts, allowing the attacker to take over the account and change student marks.	Lock the account after 5 failed attempts, add rate limiting, enforce 12-character passwords, and offer MFA for teachers.

P2 Submit Assignment (Process)

Description: Handles student assignment uploads and routes the files to storage.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Student alters assignment file after submission	Tampering	Medium	Open	9	A student submits an assignment, then finds a way to overwrite the file in D3 directly before the teacher downloads it, replacing a poor submission with a better one.	Make D3 write-once for students, use cryptographic hashes (SHA-256) to verify file integrity upon download.

P3 Manage Grades (Process)

Description: Allows teachers to upload marks and students to view their grades.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
2	Student alters grade ID in URL	Elevation of privilege	High	Open	20	A student notices their student_id=101 in the browser address bar and changes it to student_id=102. The system lacks proper authorization checks, allowing the student to view another student's grades.	Implement strict server-side access control checks (ensure the logged-in user matches the requested ID) and use indirect object references.
4	Result-day traffic floods portal	Denial of service	High	Open	15	On result day, 5,000 students try to access the portal at the exact same time. The web server is overwhelmed, crashes, and legitimate students cannot view their grades.	Implement rate limiting, use a load balancer with auto-scaling, and cache static results pages.

P4 Manage Accounts (Process)

Description: Allows admins to create users, reset passwords, and manage roles.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
7	Admin account has default password	Spoofing	Low	Mitigated	5	The test server has a default admin account (admin/admin). If this configuration is copied to production, an attacker can easily guess the credentials.	Force a password change on first login and enforce a complex password policy.

session token (Data Flow)

Properties: Encrypted

Number	Title	Type	Severity	Status	Score	Description	Mitigations
12	Session token stolen on the network	Information disclosure	High	Open	16	A student logs in on a public university Wi-Fi network without HTTPS. An attacker sniffs the network traffic, steals the session token, and hijacks the student's active session.	Enforce TLS (HTTPS) on all data flows, set secure and HttpOnly flags on cookies.

grade query (Data Flow)

Description: Student requesting to view their specific grades.

Properties: Encrypted

Number	Title	Type	Severity	Status	Score	Description	Mitigations
10	SQL injection in grade search	Tampering	High	Open	20	A student enters ' OR 1=1 -- into the grade query field. The application concatenates this directly into an SQL query, returning all grades in the database.	Use parameterized queries (prepared statements) in P3 to separate data from code, and implement input validation.

payment confirmation (Data Flow)

Description: Transaction success/failure response returned by the gateway.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
11	Fake payment confirmation sent	Tampering	High	Open	20	The payment gateway sends fee confirmations to P5 without a cryptographic signature. An attacker intercepts the flow and sends a fake "success" response, allowing a student to bypass fee payment.	Require cryptographic signatures on all payment callbacks and verify the signature in P5 before updating the database.

P6 Generate Reports (Process)

Description: Compiles student data and results into reports for the Admin.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
9	Report generation crashes server	Denial of service	Low	Open	6	An Admin requests a massive report for all students across all semesters. The query consumes all server memory, causing the portal to go offline for other users.	Run heavy report generation in a background queue, limit the date range for reports, and implement timeout limits.

D1 User Accounts DB (Store)

Description: Stores user credentials, roles, and profile information.

Properties: Stores Credentials

Number	Title	Type	Severity	Status	Score	Description	Mitigations
5	Unencrypted backup of user DB	Information disclosure	Medium	Open	12	The database administrator takes a nightly backup of D1 and stores it on a shared network drive without encryption. An attacker gains access to the drive and steals all user credentials.	Encrypt database backups at rest, store them in a secure offline location, and restrict access.

D3 Assignment File Storage (Store)

Description: Stores uploaded student assignment files.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
6	Uploaded file contains malware	Tampering	Medium	Open	8	A malicious student uploads a PDF assignment that contains a hidden executable virus. When the teacher downloads the file to grade it, the malware infects the teacher's computer.	Scan all uploaded files with antivirus software, restrict file types to PDF/DOCX, and disable macros.

D4 Audit Log (Store)

Description: Stores system logs, failed logins, and transaction history.

Properties: Is a Log

Number	Title	Type	Severity	Status	Score	Description	Mitigations
3	Marks changed without an audit trail	Repudiation	Medium	Open	10	A teacher changes a student's grade directly in the database but denies doing so. Because the system does not log database changes or timestamps, the university cannot prove who made the change.	Implement a write-only Audit Log (D4) that records all grade changes with the user ID, timestamp, and old/new values.

5. Risk Register

ID	Asset	Threat (STRIDE)	Vulnerability	L	I	Score	Level	Mitigation	Owner
R1	Teacher accounts	Spoofing: Brute-force on P1	No lockout, weak password rule	4	4	16	High	Lockout, rate limit, MFA	Amina
R2	Session tokens	Info Disclosure: Stolen on network	No HTTPS/TLS encryption	4	4	16	High	Enforce TLS, secure cookies	Salika
R3	Student grades	Elevation of Privilege: Alter URL	Missing server-side auth checks	4	5	20	Critical	Server-side access control	Iram
R4	Audit logs	Repudiation: Marks changed	No database logging or timestamps	2	5	10	Medium	Write-only Audit Log with timestamp	Amina
R5	Portal uptime	Denial of Service: Result day flood	No rate limiting or auto-scaling	3	5	15	High	Rate limiting, load balancer	Salika
R6	Payment data	Tampering: Fake confirmation	No cryptographic signatures	4	5	20	Critical	Require cryptographic signatures	Iram
R7	Grades DB (D2)	Tampering: SQL Injection	Unsanitized input in grade query	4	5	20	Critical	Parameterized queries	Amina
R8	User DB (D1)	Info Disclosure: Unencrypted backup	Backups stored in plain text	3	4	12	High	Encrypt backups at rest	Salika

R9	File Storage (D3)	Tampering: Malware upload	No antivirus/file type checks	2	4	8	Medium	Antivirus scan, restrict file types	Iram
R10	Admin accounts	Spoofing: Default password	Test server config copied to prod	1	5	5	Low	Force password change	Amina
R11	Assignment files	Tampering: Alter after submission	No file integrity checks	3	3	9	Medium	Write-once storage, SHA-256 hashes	Salika
R12	Web server	Denial of Service: Report crash	No memory limits on report generation	2	3	6	Low	Background queues, timeout limits	Iram

6. SDLC Mapping and Analysis

Table: Risk-to-SDLC mapping

Risk ID	Threat	SDLC Phase	Security Activity	Objective
R1	Brute-force on P1	Design, Implementation, Testing	Define lockout rule, code it, test it	Authenticity
R2	Session token theft	Design, Deployment	TLS on all flows, secure cookies	Confidentiality
R3	Grade ID tampering	Requirements, Testing	Access-control rule, test cases	Confidentiality
R4	No audit trail	Design, Implementation	Write audit log with timestamps	Accountability
R5	Result-day flood	Deployment, Maintenance	Rate limit, autoscaling, monitoring	Availability

R6	Forged payment callback	Design, Implementation	Signed callbacks, verify at P5	Integrity
R7	SQL injection	Implementation, Testing	Parameterized queries, static analysis	Integrity
R8	Unencrypted DB backup	Deployment, Maintenance	Encrypt backups, restrict access	Confidentiality

Analysis:

1. The Implementation and Design phases have the most security activities in our table. This is because flaws like SQL injection, missing access controls, and bad architecture are cheapest to prevent before the code is even written.
2. The Confidentiality objective is under the most threat in SmartCampus. This is because the system handles highly sensitive data, including student grades, login credentials, and session tokens.
3. The SQL Injection (R7) threat would have been extremely expensive to fix after release. If an attacker exploited this and wiped the Grades DB, the university would face massive data loss, reputational damage, and legal issues. Fixing it during implementation (using parameterized queries) takes minutes; fixing it after a breach takes months.

7. GitHub Work

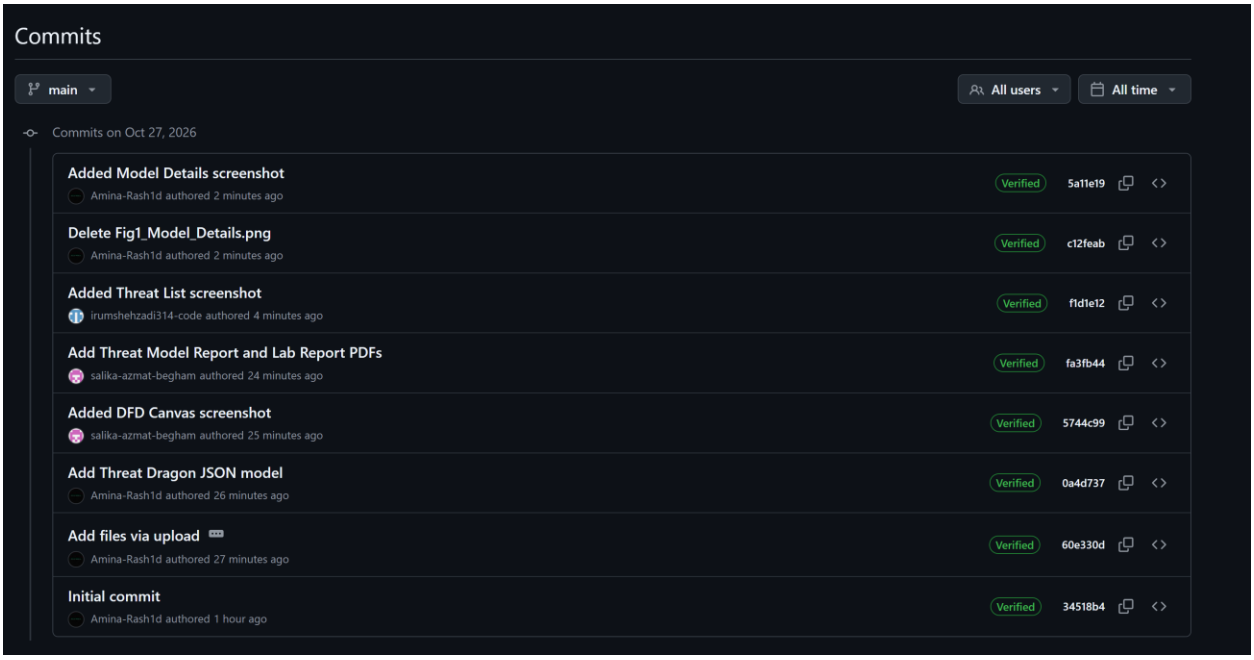


Figure 4: *GitHub commit history showing contributions from all group members (Amina, Iram, and Salika).*

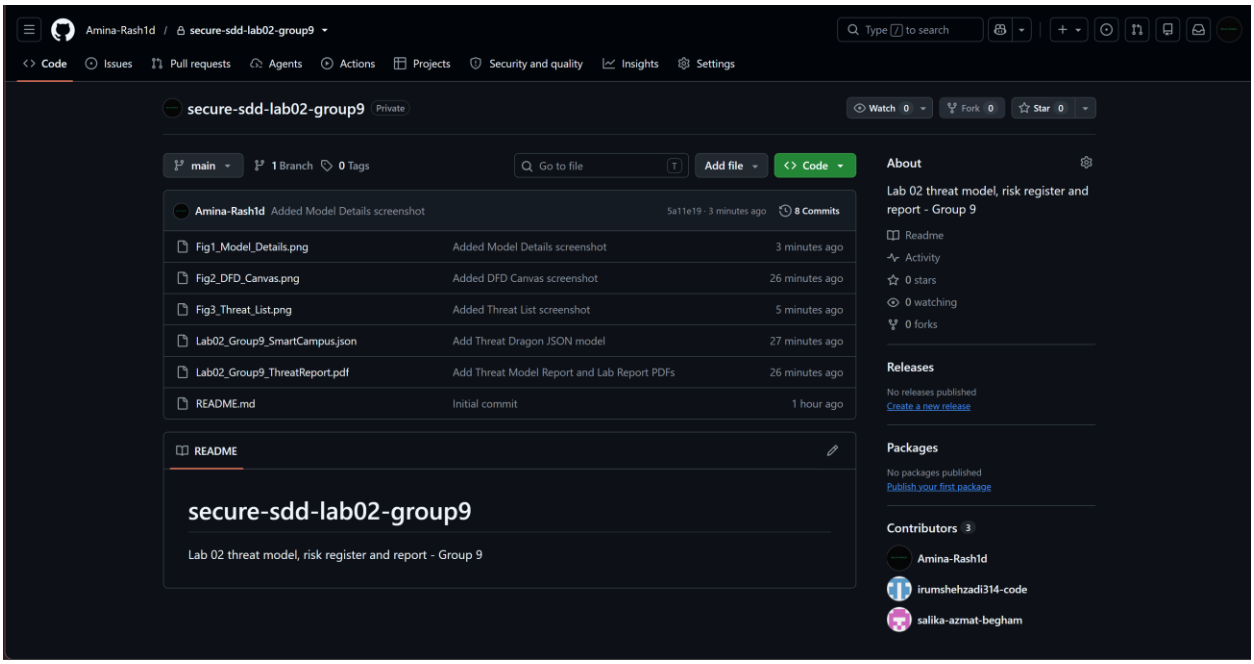


Figure 5: *GitHub repository structure showing the uploaded files.*

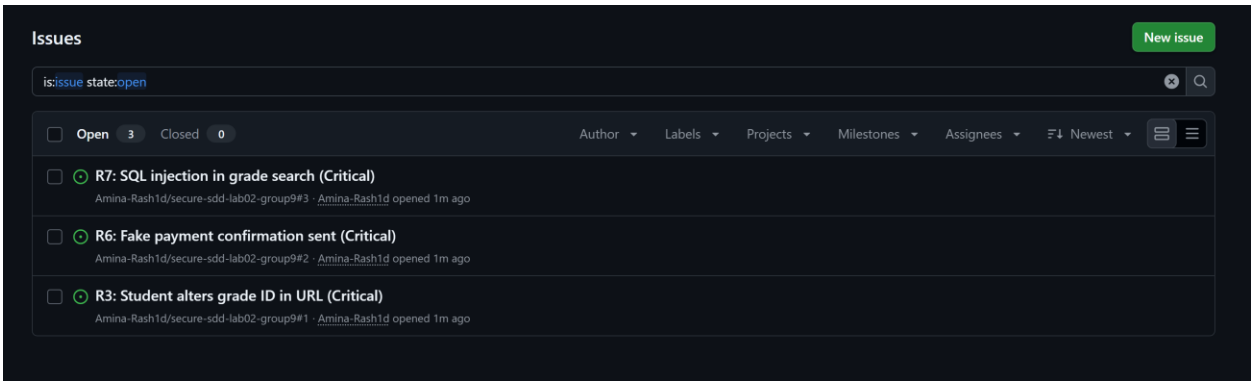


Figure 6: *GitHub Issues tracking the Critical risks.*

Repository Link: https://github.com/Amina-Rash1d/secure-sdd-lab02-group9

7. Answers to Practice Questions

1. List 10 assets of SmartCampus and rate each one for confidentiality, integrity and availability (High, Medium, Low).

Asset	Confidentiality (C)	Integrity (I)	Availability (A)
1. User Passwords	High	High	Medium
2. Student Grades	High	High	Medium
3. Session Tokens	High	High	Low
4. Assignment Files	Medium	High	Medium
5. Audit Logs	Low	High	Medium
6. Payment Confirmations	High	High	Low
7. Portal Uptime	Low	Low	High
8. Admin OTPs	High	High	Low
9. Teacher Notes	Medium	Medium	Low
10. Web Server Config	Medium	High	High

2. Find at least one threat for each STRIDE letter and place it on the DFD element where it applies.

- **Spoofing:** Attacker guesses a teacher's password on **P1 Authenticate User**.
- **Tampering:** Student injects SQL code into the grade search on the **Flow: grade query**.
- **Repudiation:** Teacher changes a mark in **D4 Audit Log** but denies doing it.
- **Information Disclosure:** Attacker sniffs the network to steal a session on the **Flow: session token**.
- **Denial of Service:** 5,000 students flood the portal on result day, crashing **P3 Manage Grades**.
- **Elevation of Privilege:** Student alters their ID in the URL to view others' grades on **P3 Manage Grades**.

3. A student can change student_id=101 to student_id=102 in the browser address bar and see another student's grades. Name the STRIDE type, the vulnerability, the OWASP category, the likelihood and impact scores, the risk level and two controls.

- **STRIDE Type:** Elevation of Privilege
- **Vulnerability:** Insecure Direct Object Reference (IDOR) / Missing server-side authorization checks.
- **OWASP Category:** A01:2021 - Broken Access Control.
- **Likelihood:** 4 (Likely)
- **Impact:** 5 (Severe)
- **Risk Level:** 20 (Critical)
- **Controls:**
    1. Implement strict server-side access control checks to verify the logged-in user matches the requested ID.
    2. Use indirect object references (like UUIDs) instead of sequential database IDs in the URL.

4. The payment gateway sends fee confirmations to P5 without a signature. Describe the threat, the trust boundary crossed, and a mitigation.

- **Threat:** An attacker intercepts the communication and sends a fake "success" payment confirmation (Tampering).
- **Trust Boundary Crossed:** TB3 (Third-party Services).
- **Mitigation:** Require cryptographic signatures (like HMAC) on all payment callbacks and verify the signature in P5 before marking the fee as paid.

5. On result day 5,000 students open the portal at once and it slows to a crawl. Which security objective is hit? Give two design-phase and two deployment-phase controls.

- **Security Objective Hit:** Availability.
- **Design-Phase Controls:** Implement a load balancer architecture and design the system for auto-scaling.
- **Deployment-Phase Controls:** Configure rate limiting on the web server to block excessive requests from single IPs, and use a Content Delivery Network (CDN) to cache static result pages.

6. The admin account has the same password as its username on the test server, and the test server is copied to production. Which SDLC phase failed and what activity would have caught it?

- **SDLC Phase Failed:** Deployment (or Release/Configuration Management).
- **Activity that would have caught it:** Security Hardening / Secure Configuration Review. Before pushing to production, a checklist should have been run to ensure default or weak credentials were removed and disabled.

7. A teacher says they never changed a mark but the Grades DB shows a change. Which STRIDE type is this and which asset (D4) must exist to solve the dispute?

- **STRIDE Type:** Repudiation.
- **Asset Required:** The **D4 Audit Log** asset must exist. It needs to record the user ID, timestamp, and old/new values of every grade change to prove who made the modification.

8. A new feature, a mobile app, is planned for next year. Explain which parts of the threat model you would repeat and when.

- If a mobile app is added, we must repeat the **Design phase (STRIDE modelling)**. A mobile app introduces entirely new threat vectors, such as insecure local data storage on the phone, reverse engineering of the app, and different authentication

flows. We would draw a new DFD for the mobile app architecture and run STRIDE again. We would also repeat the **Testing phase** to include mobile-specific penetration testing.

9. Conclusion

Through this lab, we successfully used OWASP Threat Dragon to model security risks for the SmartCampus portal. By applying the STRIDE methodology, we identified 12 critical security threats across the data flows, processes, and data stores. The hardest part was ensuring that the data flows accurately crossed the trust boundaries, as this required careful mapping of the system architecture. We learned that identifying threats during the design phase allows developers to build in security controls (like parameterized queries and TLS) before any code is written, saving time and preventing major data breaches. The GitHub repository allowed us to track these risks as manageable issues for the development team.

10. Role Sign-off Lines

- **Group Lead / Facilitator:** Amina Rashid
- **Asset and Boundary Analyst:** Salika Bibi
- **Threat Analyst:** Iram Shehzadi